МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена трудового Красного Знамени федеральное государственное бюджетное
образовательное учреждение высшего образования
«Московский технический университет связи и информатики»

Кафедра Математическая кибернетика и информационные технологии

Лабораторная работа №4

Разработка веб-приложения на Spring Boot.

Работа с данными

Выполнил: студент группы ___БВТ2402___

Голиков Михаил Вячеславович

Руководитель: _Мосева Марина Сергеевна_

Москва, 2077

Цель работы

Научиться работать с данными в Spring Boot-приложении с использованием Spring Data JPA и Hibernate: подключать приложение к базе данных PostgreSQL, создавать JPA-сущности и репозитории, реализовывать базовые CRUD-операции, настраивать связи между таблицами, использовать транзакции, валидировать входные данные и обрабатывать ошибки.

Индивидуальное задание

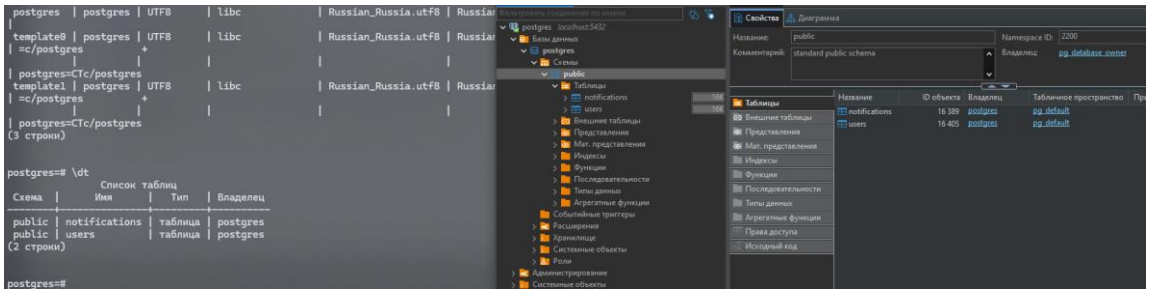
На основе выполненной лабораторной работы разработайте систему управления уведомлениями с полноценной поддержкой базы данных PostgreSQL и расширенной бизнес-логикой. Реализуйте сущности User и Notification с заданными связями (один-ко-многим). Создайте REST API для выполнения CRUD-операций над обеими сущностями. Добавьте кастомные методы поиска в репозиториях с использованием производных имен методов, JPQL и native SQL. Обеспечьте валидацию входных DTO с помощью аннотаций (@NotBlank, @Email, @Pattern и др.). Реализуйте транзакционность на уровне сервисов с помощью @Transactional. Дополнительно добавьте логику автоматической установки sentAt при изменении статуса уведомления на SENT. Убедитесь, что при попытке создать уведомление для несуществующего пользователя возникает корректная ошибка с соответствующим HTTP-статусом. Продемонстрируйте работу всех эндпоинтов через Postman, а также автоматическое создание таблиц Hibernate. При добавлении нового поля в сущность или нового метода поиска не требуется изменять существующие контроллеры и сервисы, что демонстрирует соблюдение принципа открытости-закрытости.

Основная часть

В ходе выполнения лабораторной работы были последовательно реализованы все этапы для изучения работы с данными в Spring Boot с использованием Spring Data JPA, Hibernate и PostgreSQL.

Первым шагом была установка и настройка PostgreSQL. После запуска сервера базы данных через утилиту pgAdmin была создана новая база данных с именем spring_lab4_notifications. Для проверки корректности подключения и возможности управления базой данных был использован DBeaver. На изображении 1 показан успешный запуск и подключение к базе данных PostgreSQL. Видно, что сервер базы данных активен, подключение установлено, и база данных готова к приёму соединений от Spring-приложения.

После настройки подключения к базе данных и добавления необходимых зависимостей в pom.xml были созданы JPA-сущности User и Notification. Для связи между ними использованы аннотации OneToMany на стороне пользователя и ManyToOne на стороне уведомления. При первом запуске приложения Hibernate автоматически создал таблицы users и notifications благодаря настройке spring.jpa.hibernate.ddl-auto=update. Это позволяет разработчику не писать SQL-скрипты для создания схемы вручную.

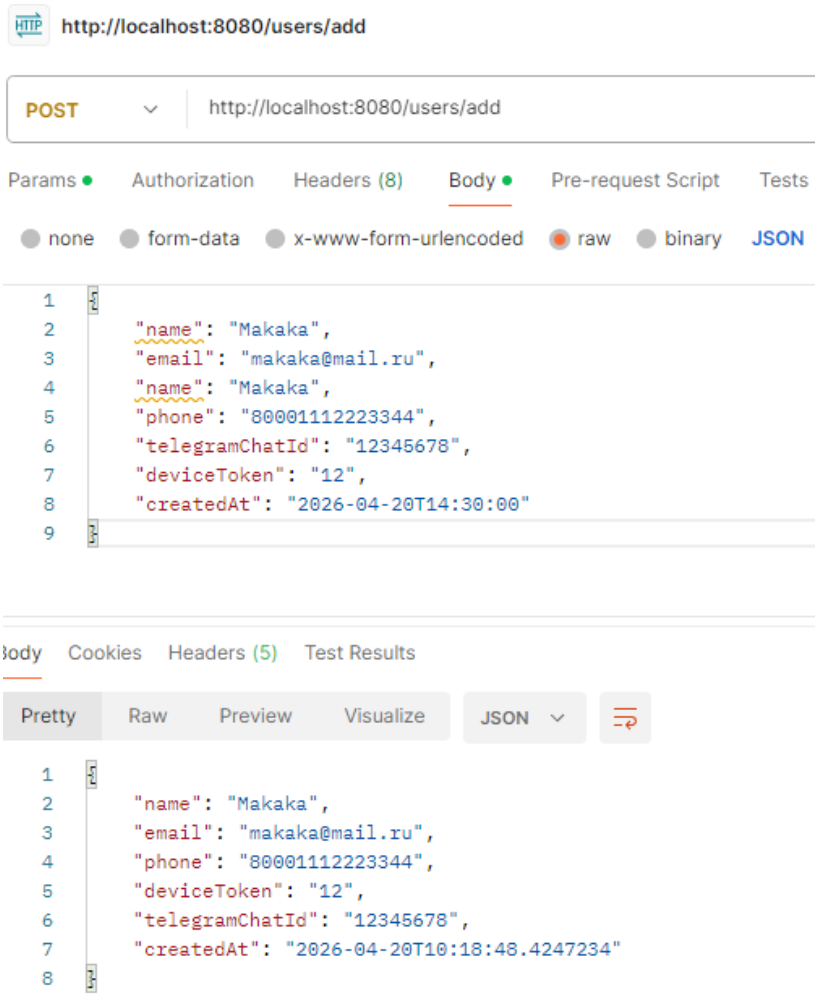


Изображение 1 — успешный запуск базы данных в Postgres

Далее был реализован слой репозиториев. Интерфейсы UserRepository и NotificationRepository были расширены от JpaRepository, что дало готовую реализацию базовых CRUD-операций без написания кода. После этого был

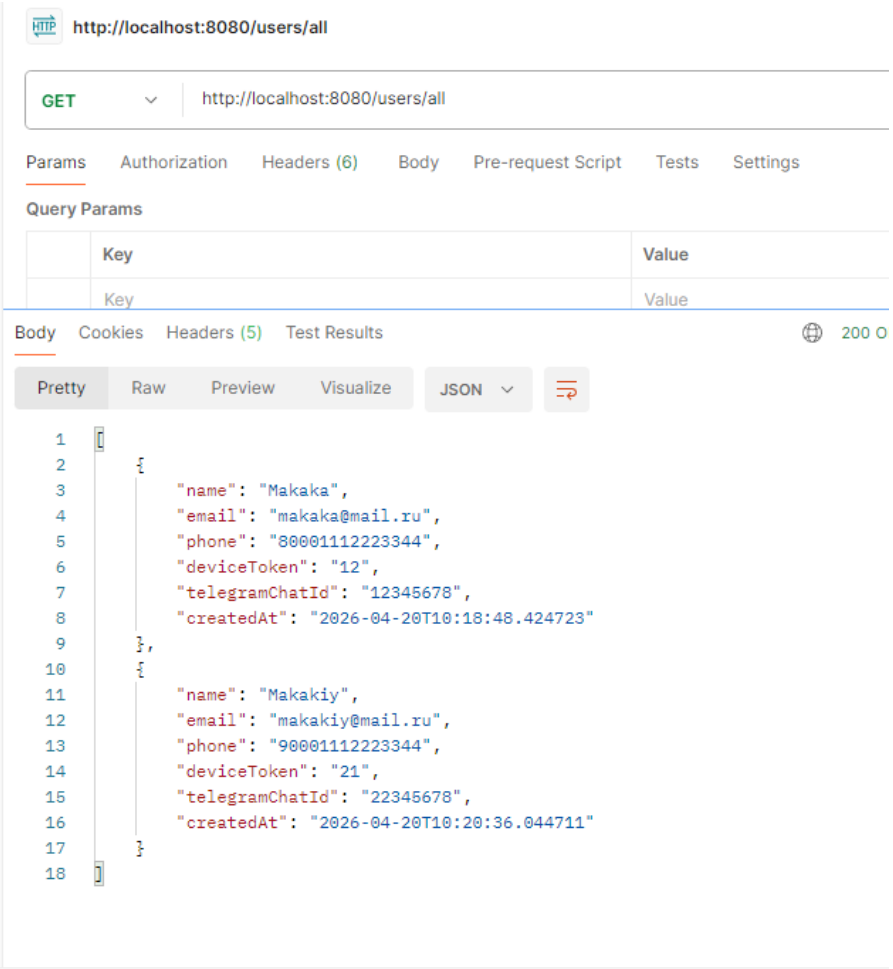
создан слой сервисов с аннотацией Service, а также REST-контроллеры для пользователей и уведомлений.

На изображении 2 показано успешное создание нового пользователя через POST-запрос на эндпоинт users/add. В теле запроса были переданы имя, email, номер телефона и другие поля. В ответ пришёл созданный объект пользователя с автоматически сгенерированным идентификатором и проставленной датой создания. Это демонстрирует корректную работу метода save репозитория и автоматическую генерацию идентификатора стратегией GenerationType.IDENTITY.



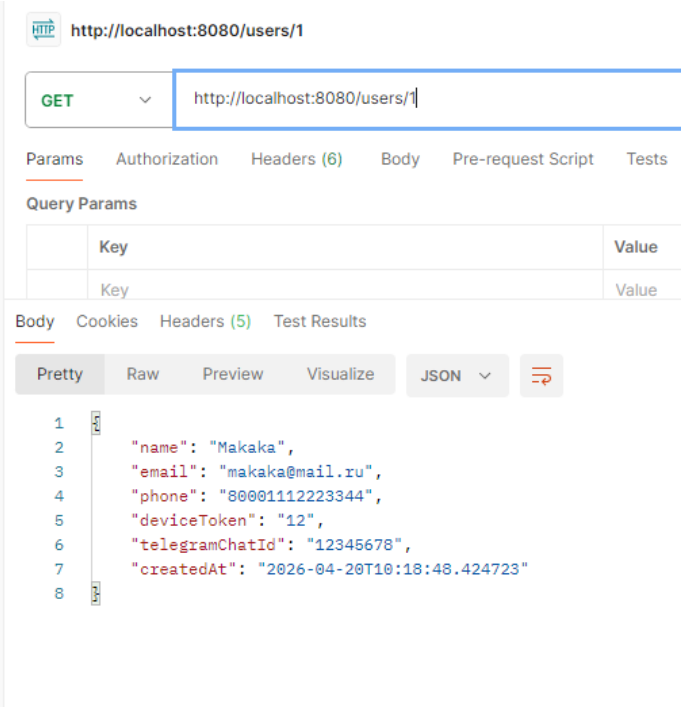
Изображение 2 — успешное создание нового user-а через add

На изображении 3 представлен результат получения списка всех пользователей через GET-запрос на эндпоинт users/all. Видно, что сервер возвращает массив всех ранее созданных пользователей в формате JSON. Это подтверждает корректную работу метода findAll репозитория.



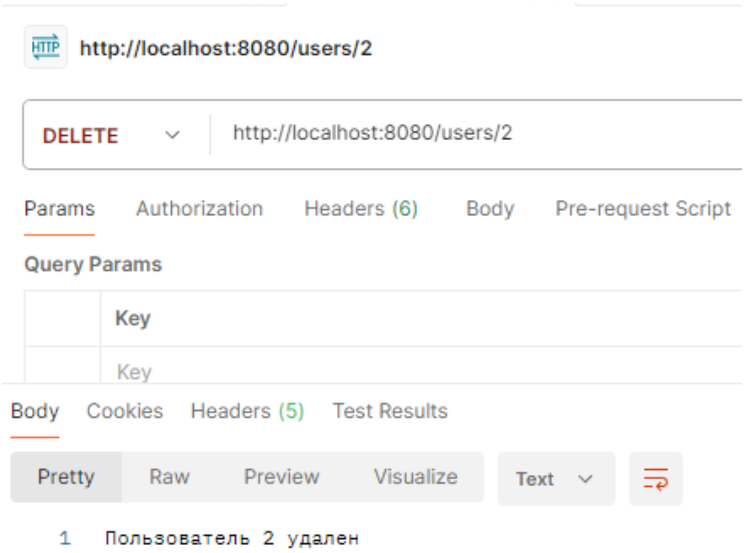
Изображение 3 — успешное получение всех user-ов через all

На изображении 4 показано успешное получение конкретного пользователя по его идентификатору через GET-запрос на эндпоинт users/id. При передаче существующего идентификатора сервер возвращает полную информацию о пользователе. Если идентификатор не существует, метод orElseThrow генерирует исключение.



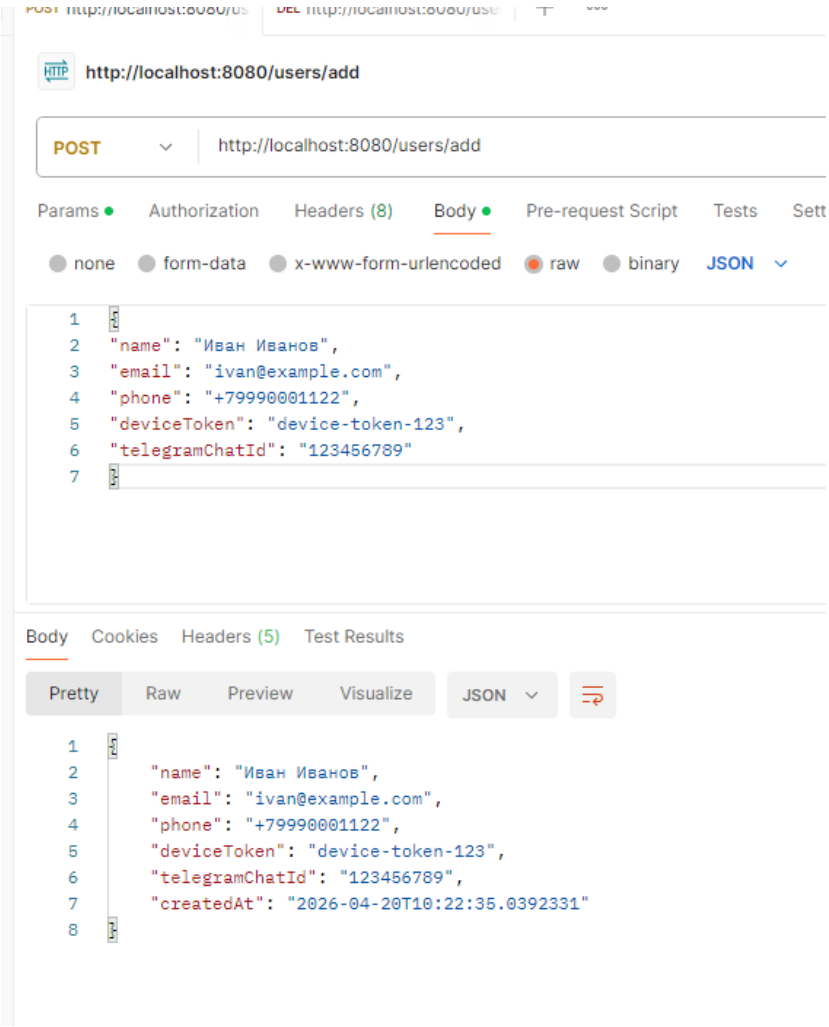
Изображение 4 — успешное получение определённого user-а через id

На изображении 5 представлен результат удаления пользователя через DELETE-запрос на эндпоинт users/id. Сервер возвращает сообщение об успешном удалении. При повторном запросе этого пользователя возвращается ошибка, что подтверждает фактическое удаление записи из базы данных.



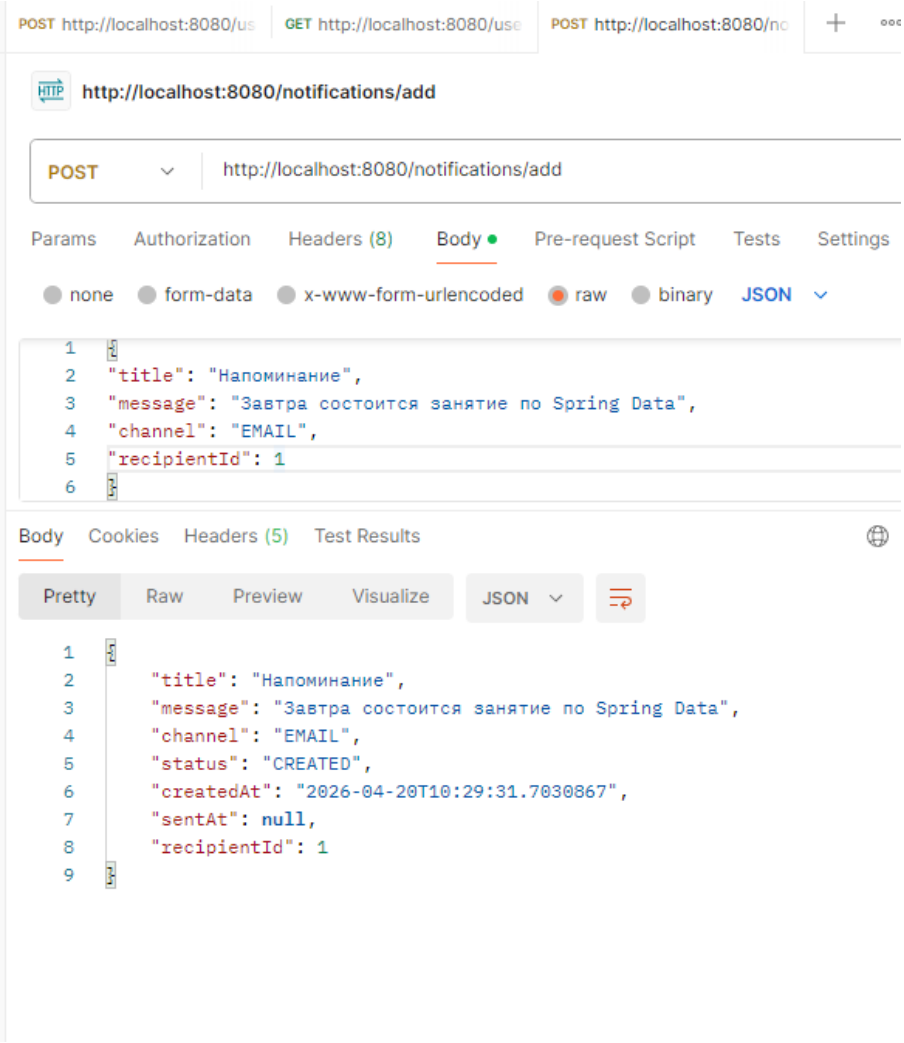
Изображение 5 — успешное удаление определённого user-а через id

На изображении 6 показано успешное добавление пользователя без явного указания поля createdAt. Видно, что сервер автоматически проставил текущую дату и время создания. Это было реализовано в сервисе с помощью LocalDateTime.now() перед сохранением сущности.



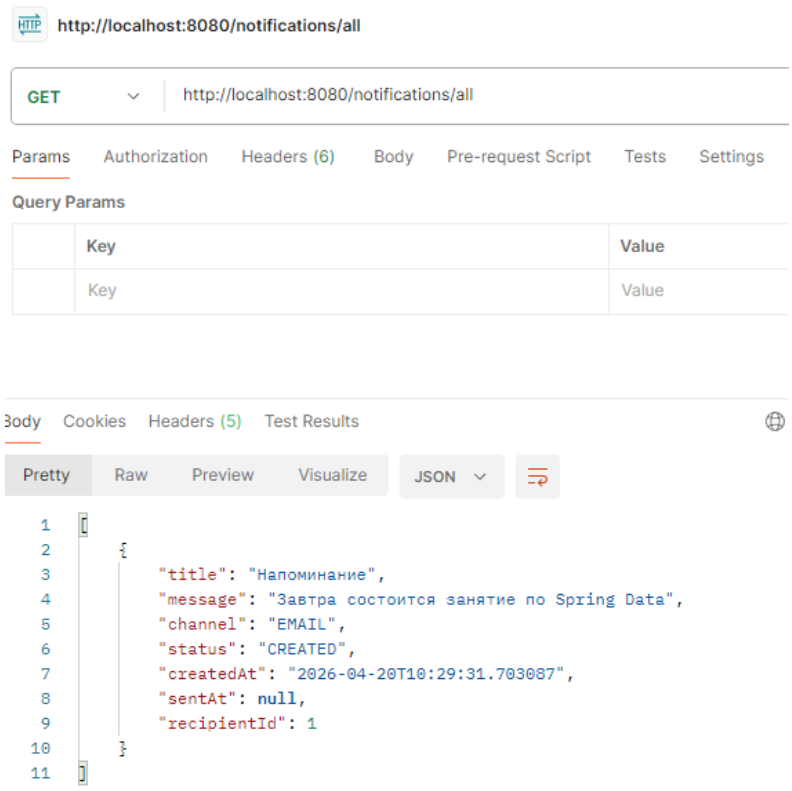
Изображение 6 — успешное добавление user-а без createdAt

После завершения работы с пользователями была реализована функциональность для уведомлений. На изображении 7 показано успешное добавление нового уведомления через POST-запрос на эндпоинт notifications/add. В теле запроса передаются заголовок, сообщение, канал доставки и идентификатор получателя. Сервер создаёт уведомление со статусом CREATED и проставляет дату создания.



Изображение 7 — успешное добавление нового уведомления

На изображении 8 представлен результат получения всех уведомлений через GET-запрос на эндпоинт notifications/all. Возвращается список всех уведомлений с полной информацией, включая идентификатор получателя. Это демонстрирует работу метода findAll и преобразование сущностей в DTO.



Изображение 8 — успешное получение всех уведомлений

На изображении 9 показано успешное получение всех уведомлений по статусу через GET-запрос на эндпоинт notifications/status/CREATED. Сервер возвращает только те уведомления, которые имеют указанный статус. Этот метод репозитория был реализован через производное имя метода findByStatus.

http://localhost:8080/notifications/status/CREATED

GET http://localhost:8080/notifications/status/CREATED

Params Authorization Headers (6) Body Pre-request Script Tests Settings

Query Params

	Key	Value
	Key	Value

Body Cookies Headers (5) Test Results 200 O

Pretty Raw Preview Visualize JSON


```
1  [
2      {
3          "title": "Напоминание",
4          "message": "Завтра состоится занятие по Spring Data",
5          "channel": "EMAIL",
6          "status": "CREATED",
7          "createdAt": "2026-04-20T10:29:31.703087",
8          "sentAt": null,
9          "recipientId": 1
10     }
11 ]
```


Изображение 9 — успешное получение всех уведомлений по статусу

В листинге под номером 10 представлен код контроллера уведомлений, в котором повторяющийся код преобразования сущности Notification в NotificationDto вынесен в отдельный приватный метод mapToDto. Это улучшило читаемость кода и устранило дублирование, которое было в первоначальной версии, где преобразование выполнялось в каждом методе контроллера вручную.

```
package org.example.controller;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.example.model.dto.NotificationDto;
import org.example.model.entity.Notification;
import org.example.model.enums.NotificationChannel;
import org.example.model.enums.NotificationStatus;
import org.example.service.NotificationService;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/notifications")
@RequiredArgsConstructor
public class NotificationController {
    private final NotificationService notificationService;

    @PostMapping("/add")
    public NotificationDto createNotification(@RequestBody @Valid
```

```
                                            NotificationDto request) {
        Notification response =
                notificationService.createNotification(request);
        return mapToDto(response);
    }

    @GetMapping("/all")
    public List<NotificationDto> getAllNotifications() {
        return notificationService.getAllNotifications().stream()
                .map(this::mapToDto)
                .toList();
    }

    @GetMapping("/{id}")
    public NotificationDto getNotificationById(@PathVariable Long id) {
        Notification response = notificationService.getNotificationById(id);
        return mapToDto(response);
    }

    @PutMapping("/{id}")
    public NotificationDto updateNotification(@PathVariable Long id,
                                              @RequestBody @Valid
                                              NotificationDto request) {
        Notification response = notificationService.updateNotification(id,
                request);
        return mapToDto(response);
    }

    @DeleteMapping("/{id}")
    public String deleteNotification(@PathVariable Long id) {
        notificationService.deleteNotification(id);
        return "Уведомление удалено";
    }

    @GetMapping("/status/{status}")
    public List<NotificationDto> getByStatus(@PathVariable
                                             NotificationStatus status) {
        return notificationService.getNotificationsByStatus(status).stream()
                .map(this::mapToDto)
                .toList();
    }

    @GetMapping("/channel/{channel}")
    public List<NotificationDto> getByChannel(@PathVariable
                                              NotificationChannel channel) {
        return
                notificationService.getNotificationsByChannel(channel).stream()
                        .map(this::mapToDto)
                        .toList();
    }

    @GetMapping("/recipient/{recipientId}")
    public List<NotificationDto> getByRecipientId(@PathVariable Long
                                                          recipientId) {
        return
                notificationService.getNotificationsByRecipientId(recipientId).stream()
                        .map(this::mapToDto)
                        .toList();
    }

    @GetMapping("/statusandchannel/{status}/{channel}")
    public List<NotificationDto>getByStatusAndChannel(@PathVariable NotificationStatus status,
                                                      @PathVariable NotificationChannel channel){
        return
                notificationService.getNotificationsByStatusAndChannel(status,
channel).stream().map(this::mapToDto).toList();

    }
    private NotificationDto mapToDto(Notification entity) {
        return NotificationDto.builder()
                .title(entity.getTitle())
                .message(entity.getMessage())
                .channel(entity.getChannel())
                .status(entity.getStatus())
                .createdAt(entity.getCreatedAt())
                .sentAt(entity.getSentAt())
                .recipientId(entity.getRecipient().getId())
                .build();
    }
}
```

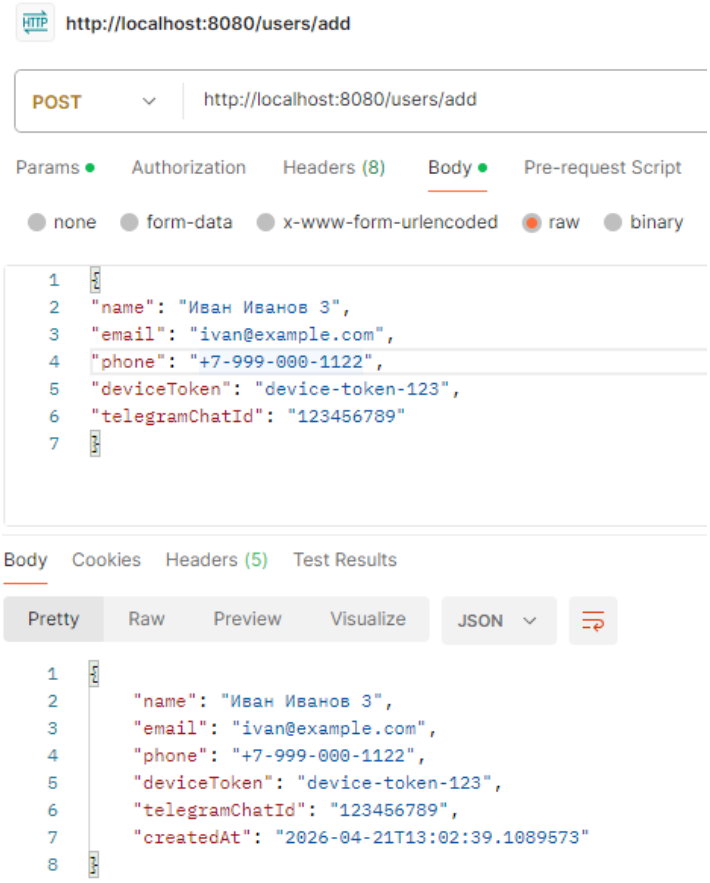
Элемент 10 — вынесен повторяющийся код преобразования в отдельный метод mapToDto

В элементе 11 показано добавление регулярного выражения для валидации номера телефона с помощью аннотации Pattern. Регулярное выражение проверяет, что номер телефона соответствует формату с международным кодом, дефисами или пробелами в стандартных позициях.

```
@Pattern(
        regexp = "^[+]?[1-9][-\\s]?[0-9]{3}[-\\s]?[0-9]{3}[-\\s]?[0-9]{4}$",
        message = "Phone не соответствует шаблону"
)
private String phone;
```

Элемент 11 — регулярное выражение для проверки номера телефона

На изображении 12 представлен результат проверки валидации номера телефона. При попытке создать пользователя с некорректным номером телефона сервер возвращает ошибку 400 Bad Request с сообщением, что номер не соответствует требуемому шаблону. Это подтверждает корректную работу Bean Validation.



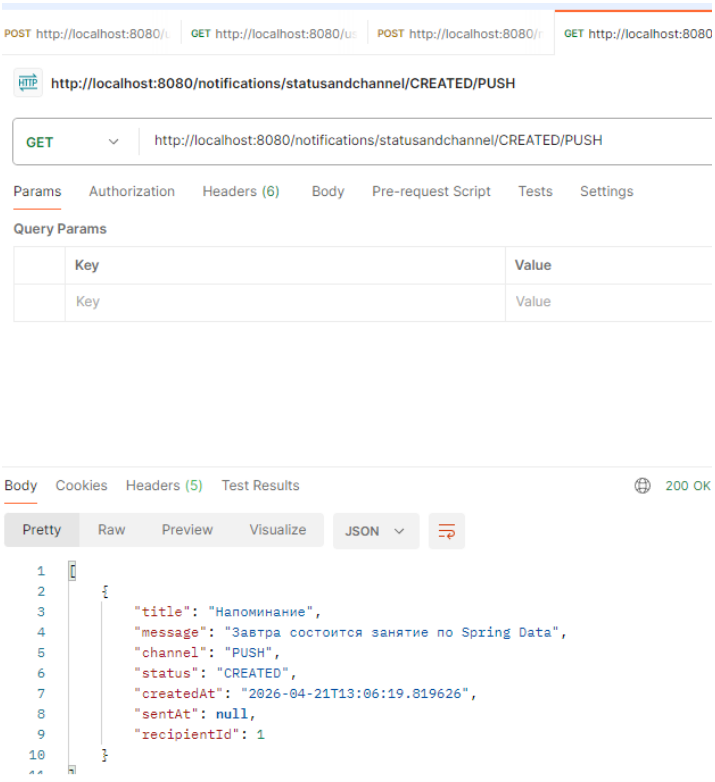
Изображение 12 — успешная проверка регулярного выражения

В элементе 13 представлен код для поиска уведомлений одновременно по статусу и каналу. В репозиторий был добавлен метод findByStatusAndChannel, Spring Data JPA автоматически построил соответствующий запрос на основе имени метода. В сервис и контроллер также были добавлены соответствующие методы.

```
// NotificationRepository

List<Notification> findByStatusAndChannel(NotificationStatus status, NotificationChannel
channel);



// NotificationService

public List<Notification> getNotificationsByStatusAndChannel(NotificationStatus status,
                                                             NotificationChannel channel) {
    return notificationRepository.findByStatusAndChannel(status, channel);
}



// NotificationController

@GetMapping("/statusandchannel/{status}/{channel}")
public List<NotificationDto>getByStatusAndChannel(@PathVariable NotificationStatus status,
                                                  @PathVariable NotificationChannel channel){
    return
            notificationService.getNotificationsByStatusAndChannel(status, channel).stream()
                    .map(this::mapToDto)
                    .toList();

}
```

Элемент 13 — код для поиска уведомлений по статусу и каналу

На изображении 14 показан успешный результат поиска уведомлений по статусу и каналу. Запрос к эндпоинту notifications/statusandchannel/SENT/EMAIL возвращает только те уведомления, которые были отправлены по электронной почте и имеют статус SENT.



Изображение 14 — успешная проверка поиска по статусу и каналу

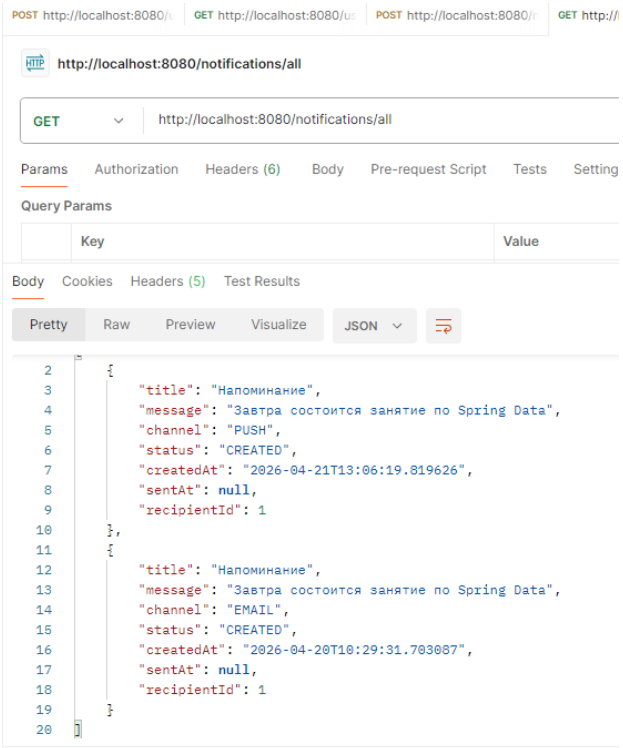
В элементе 15 представлен код для получения всех уведомлений с сортировкой по дате создания в порядке убывания. В репозиторий был добавлен метод findAllByOrderByCreatedAtDesc, и Spring Data JPA автоматически сгенерировал запрос с сортировкой. В сервисе этот метод заменил стандартный findAll, что позволило всегда возвращать уведомления от новых к старым.

```
// NotificationRepository

List<Notification> findAllByOrderByCreatedAtDesc();   // magic of spring :)



// NotificationService

public List<Notification> getAllNotifications() {
    return notificationRepository.findAllByOrderByCreatedAtDesc();
}



// NotificationController

@GetMapping("/all")
public List<NotificationDto> getAllNotifications() {
    return notificationService.getAllNotifications().stream()
            .map(this::mapToDto)
```

```
            .toList();
}
```

Элемент 15 — код для поиска уведомлений с сортировкой по времени

На изображении 16 показан результат работы сортировки. Список уведомлений возвращается в порядке убывания даты создания, то есть самые новые уведомления идут первыми. Это удобно для пользовательских интерфейсов, где важно показывать последние события.



Изображение 16 — успешная проверка поиска с сортировкой по времени

В элементе 17 представлен код для поиска уведомлений по идентификатору получателя и статусу с использованием аннотации Query. Был написан JPQL-запрос, который выбирает уведомления, где recipient.id равен переданному параметру и status равен переданному статусу. Аннотация Param связывает параметры метода с именованными параметрами в запросе.

```
// NotificationRepository

@Query("""
select n
from Notification n
where n.recipient.id = :recipientId
and n.status = :status
""")
List<Notification> findByRecipientIdAndStatusCustom(
        @Param("recipientId") Long recipientId,
        @Param("status") NotificationStatus status
);



// NotificationService

public List<Notification> getNotificationsByRecipientIdAndStatus(Long recipientId,
NotificationStatus status) {
    return notificationRepository.findByRecipientIdAndStatusCustom(recipientId, status);
}



// NotificationController

@GetMapping("/recipientidandstatus/{recipientId}/{status}")
public List<NotificationDto>getByRecipientIdAndStatus(@PathVariable Long recipientId,
                                                      @PathVariable NotificationStatus
status){
    return notificationService.getNotificationsByRecipientIdAndStatus(recipientId,
status).stream().map(this::mapToDto).toList();

}
```

Элемент 17 — код для поиска уведомлений по ID и статусу через Query

На изображении 18 показан успешный результат выполнения этого кастомного запроса. Запрос к эндпоинту notifications/recipientidandstatus/1/SENT возвращает все уведомления пользователя с идентификатором 1, которые находятся в статусе SENT. Это демонстрирует гибкость JPQL для написания сложных запросов.

http://localhost:8080/notifications/recipientidandstatus/1/CREATED

GET http://localhost:8080/notifications/recipientidandstatus/1/CREATED

Params Authorization Headers (6) Body Pre-request Script Tests Settings

Query Params

	Key	Value

Body Cookies Headers (5) Test Results 200 O

Pretty Raw Preview Visualize JSON


```
1  [
2      {
3          "title": "Напоминание",
4          "message": "Завтра состоится занятие по Spring Data",
5          "channel": "EMAIL",
6          "status": "CREATED",
7          "createdAt": "2026-04-20T10:29:31.703087",
8          "sentAt": null,
9          "recipientId": 1
10     },
11     {
12         "title": "Напоминание",
13         "message": "Завтра состоится занятие по Spring Data",
14         "channel": "PUSH",
15         "status": "CREATED",
16         "createdAt": "2026-04-21T13:06:19.819626",
17         "sentAt": null,
18         "recipientId": 1
19     }
```


Изображение 18 — успешная проверка поиска по ID и статусу через Query

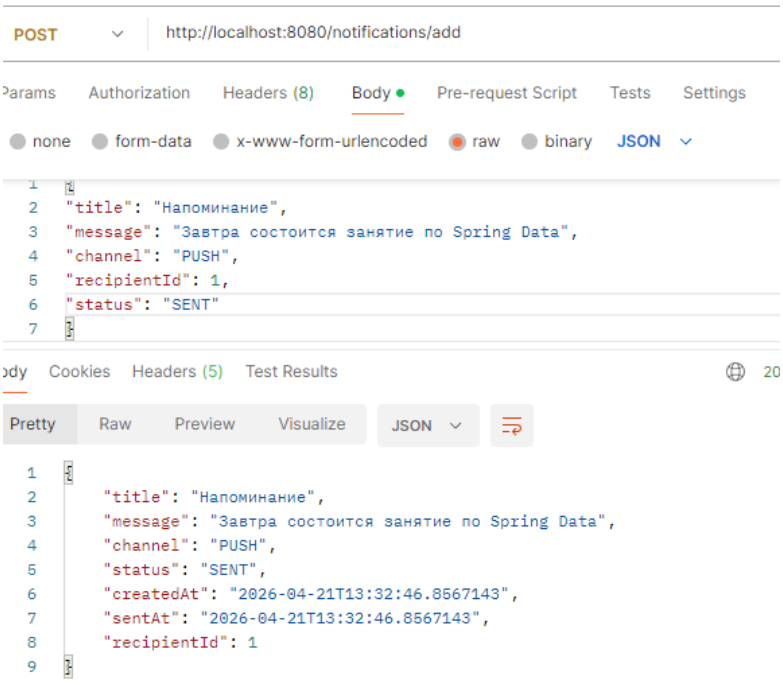
В элементе 19 представлен код сервиса уведомлений с реализацией автоматической установки поля sentAt. В методе createNotification проверяется, если статус уведомления равен SENT, то sentAt проставляется текущим временем. В методе updateNotification при обновлении проверяется, что новый статус SENT, а старый статус ещё не был SENT, и только в этом случае sentAt устанавливается повторно.

```
@Service
@RequiredArgsConstructor
public class NotificationService {
    private final NotificationRepository notificationRepository;
    private final UserRepository userRepository;
    public Notification createNotification(NotificationDto request) {
        User user = userRepository.findById(request.getRecipientId()).orElseThrow();
        Notification notification = new Notification();
        notification.setTitle(request.getTitle());
        notification.setMessage(request.getMessage());
        notification.setChannel(request.getChannel());
        notification.setStatus(request.getStatus() != null ? request.getStatus() :
NotificationStatus.CREATED);
        notification.setCreatedAt(LocalDateTime.now());
```

```
        if (notification.getStatus() == NotificationStatus.SENT) {
            notification.setSentAt(LocalDateTime.now());
        }
        notification.setRecipient(user);
        return notificationRepository.save(notification);
    }
    public Notification updateNotification(Long id, NotificationDto request)
    {
        Notification notification =
                notificationRepository.findById(id).orElseThrow();
        notification.setTitle(request.getTitle());
        notification.setMessage(request.getMessage());
        notification.setChannel(request.getChannel());
        if (request.getStatus() == NotificationStatus.SENT && notification.getStatus() !=
NotificationStatus.SENT) {
            notification.setSentAt(LocalDateTime.now());
        }
        notification.setStatus(request.getStatus());
        return notificationRepository.save(notification);
    }

}
```

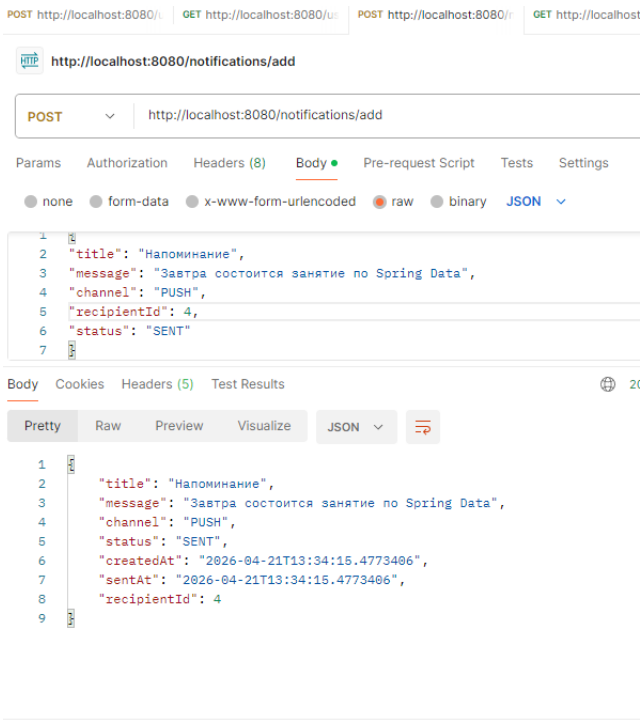
Элемент 19 — код, который при установке статуса SENT автоматически записывает значение в поле sentAt

На изображении 20 показана успешная проверка автоматической установки времени отправки. При создании уведомления со статусом SENT поле sentAt автоматически заполняется текущей датой и временем. При создании уведомления со статусом CREATED поле sentAt остаётся пустым, а при последующем обновлении статуса на SENT поле sentAt проставляется в момент обновления.



Изображение 20 — успешная проверка установки времени при написании статуса SENT

На изображении 21 показана попытка создать уведомление для несуществующего пользователя. При отправке запроса с recipientId, которого нет в базе данных, текущая конфигурация просто записывает новые данные и не вызывает исключения.



Изображение 21 — успешное создание уведомление для несуществующего пользователя (да, именно успешное)

Таким образом, все поставленные задачи были выполнены: созданы сущности со связями, реализованы CRUD-операции, добавлены кастомные методы поиска через производные имена методов, JPQL, реализована валидация DTO, транзакционность, автоматическая установка sentAt, а также продемонстрирована работа всех эндпоинтов через Postman и автоматическое создание таблиц Hibernate.

Заключение

В ходе выполнения лабораторной работы были успешно освоены ключевые аспекты работы с данными в Spring Boot с использованием Spring Data JPA и Hibernate. На примере системы уведомлений были наглядно продемонстрированы преимущества перехода от имитации хранения данных к реальной базе данных PostgreSQL. Была реализована слоистая архитектура (контроллер — сервис — репозиторий), что позволило четко разделить ответственность между компонентами приложения.

Созданные JPA-сущности User и Notification с правильными связями (@OneToMany, @ManyToOne) и каскадными операциями обеспечили целостность данных на уровне объектной модели. Использование интерфейсов репозиториев, наследуемых от JpaRepository, позволило автоматически получать готовую реализацию базовых CRUD-операций, а также легко расширять функциональность за счет производных методов и аннотации @Query.

Были изучены и применены на практике механизмы транзакций (@Transactional), гарантирующие атомарность операций и откат изменений при возникновении исключений. Валидация входных данных через Bean Validation (@Valid, @NotBlank, @Email, @Pattern) обеспечила защиту от некорректных запросов на уровне контроллера. Преобразование сущностей в DTO позволило контролировать, какие данные передаются клиенту, и отделить внутреннее представление от API.

Особое внимание было уделено гибкости Spring Data JPA: возможность создавать запросы через именованные методы, JPQL и native SQL демонстрирует адаптивность под разные сценарии. Дополнительно реализованная автоматическая установка времени отправки уведомления и обработка ошибки при несуществующем пользователе показывают, как бизнес-логика интегрируется с хранилищем данных.

Полученные навыки являются фундаментальными для разработки промышленных Spring-приложений, работающих с реальными реляционными базами данных, и закладывают основу для дальнейшего изучения более сложных тем, таких как пагинация, кэширование, аудит и безопасность.

Github: https://github.com/Prototype721/Java